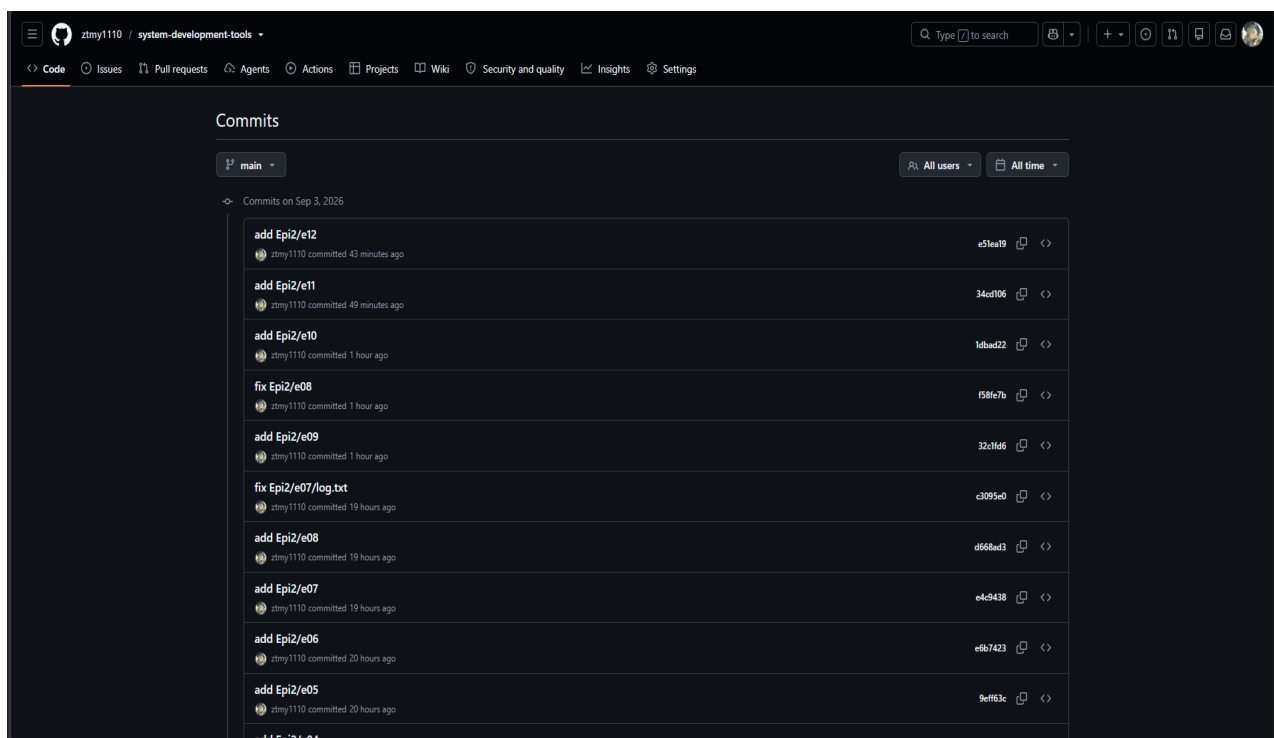


《系统开发工具基础》实验报告

姓名：夏昕 学号：25080012068
专业：软件工程 任课教师：周小伟
实验时间：2026/8/31 实验周次：第2周

GitHub 仓库链接：<https://github.com/ztmy1110/system-development-tools>

commit 记录



课堂习题

第5题：控制一个可清理的后台任务

一、主题

命令行环境：进程、信号与任务控制

二、题目要求

1. 将给定的 worker.sh 脚本保存到 q05/, 赋予脚本执行权限, 在前台启动, 并将标准输出和标准错误分别保存到 stdout.log 和 stderr.log。
2. 使用 Ctrl-Z 挂起任务, 再使用 bg 让任务在后台继续运行, 并使用 jobs 查看任务状态。
3. 使用 jobs -p 或等价方法获取任务 PID, 不能手工抄写 PID, 然后发送 SIGTERM, 等待任务正常结束。禁止使用 kill -9。
4. 检查 cleanup.log, 确认其中出现 CLEAN_EXIT, 同时确认后台任务已经退出。

三、源程序代码

```
chmod +x worker.sh
```

```
./worker.sh > stdout.log 2> stderr.log
```

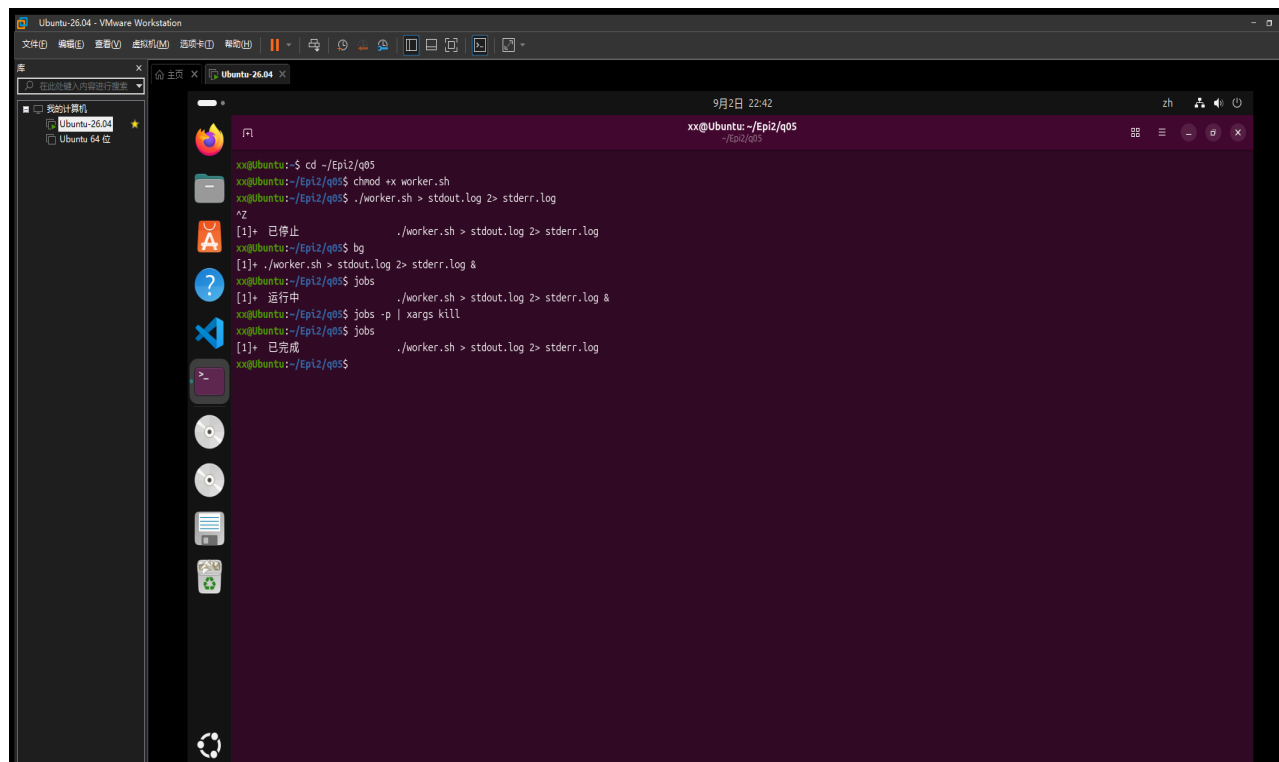
```
bg
```

```
jobs
```

```
jobs -p | xargs kill
```

```
jobs
```

四、程序运行结果



The screenshot shows a terminal window titled 'xx@Ubuntu: ~/Epi2/q05'. The user enters the following commands and receives the following output:

```
xx@Ubuntu:~$ cd ~/Epi2/q05
xx@Ubuntu:~/Epi2/q05$ chmod +x worker.sh
xx@Ubuntu:~/Epi2/q05$ ./worker.sh > stdout.log 2> stderr.log
^Z
[1]+  已停止                  ./worker.sh > stdout.log 2> stderr.log
xx@Ubuntu:~/Epi2/q05$ bg
[1]+  ./worker.sh > stdout.log 2> stderr.log &
xx@Ubuntu:~/Epi2/q05$ jobs
[1]+  运行中                  ./worker.sh > stdout.log 2> stderr.log &
xx@Ubuntu:~/Epi2/q05$ jobs -p | xargs kill
xx@Ubuntu:~/Epi2/q05$ jobs
[1]+  已完成                  ./worker.sh > stdout.log 2> stderr.log
xx@Ubuntu:~/Epi2/q05$
```

五、解题感悟

1. 这次主要熟悉了前台任务和后台任务之间的切换，之前对 Ctrl-Z、bg 和 jobs 的作用不太清楚，实际操作后比较直观。
2. 通过 jobs -p 获取 PID 比手动找进程号方便，也能减少输错 PID 的情况。
3. 这道题让我知道了程序不一定要直接强制结束，通过信号让程序自己完成清理后退出会更加合理。

第 6 题：语义重构与本地开发反馈

一、主题

开发环境与工具

二、题目要求

1. 在已经配置好 Python、pytest 和 ruff 的课程环境中打开 q06，并启用 Python 语言服务器。
2. 分别使用编辑器的跳转到定义和查找引用功能，查看 total_price 的定义和使用位置。

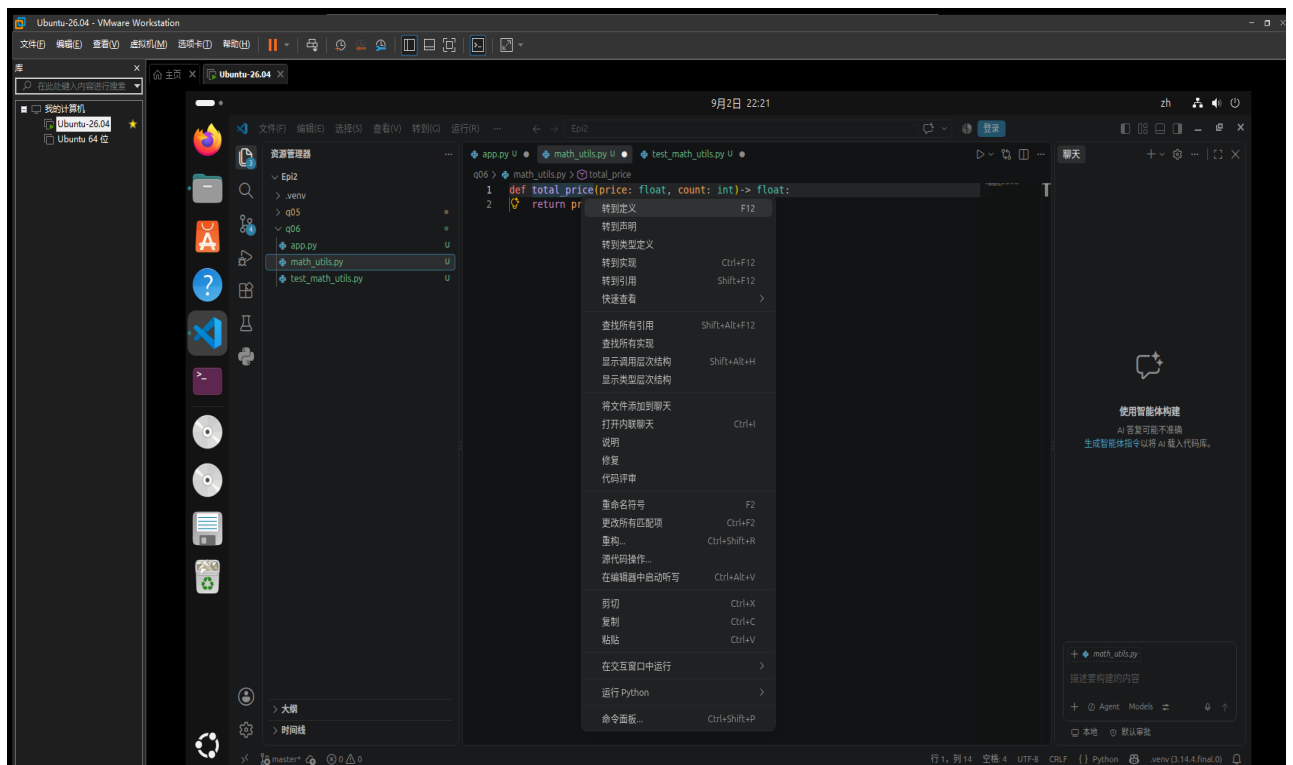
3. 使用“重命名符号”功能，将 `total_price` 修改为 `calculate_total`，保证定义和所有引用同步修改。
4. 在 `app.py` 中临时加入未使用的 `import os`，使用 `ruff` 检查并发现问题，然后删除该无用导入。
5. 最后运行 `ruff` 和 `pytest`，保证代码检查和测试都通过。

三、源程序代码

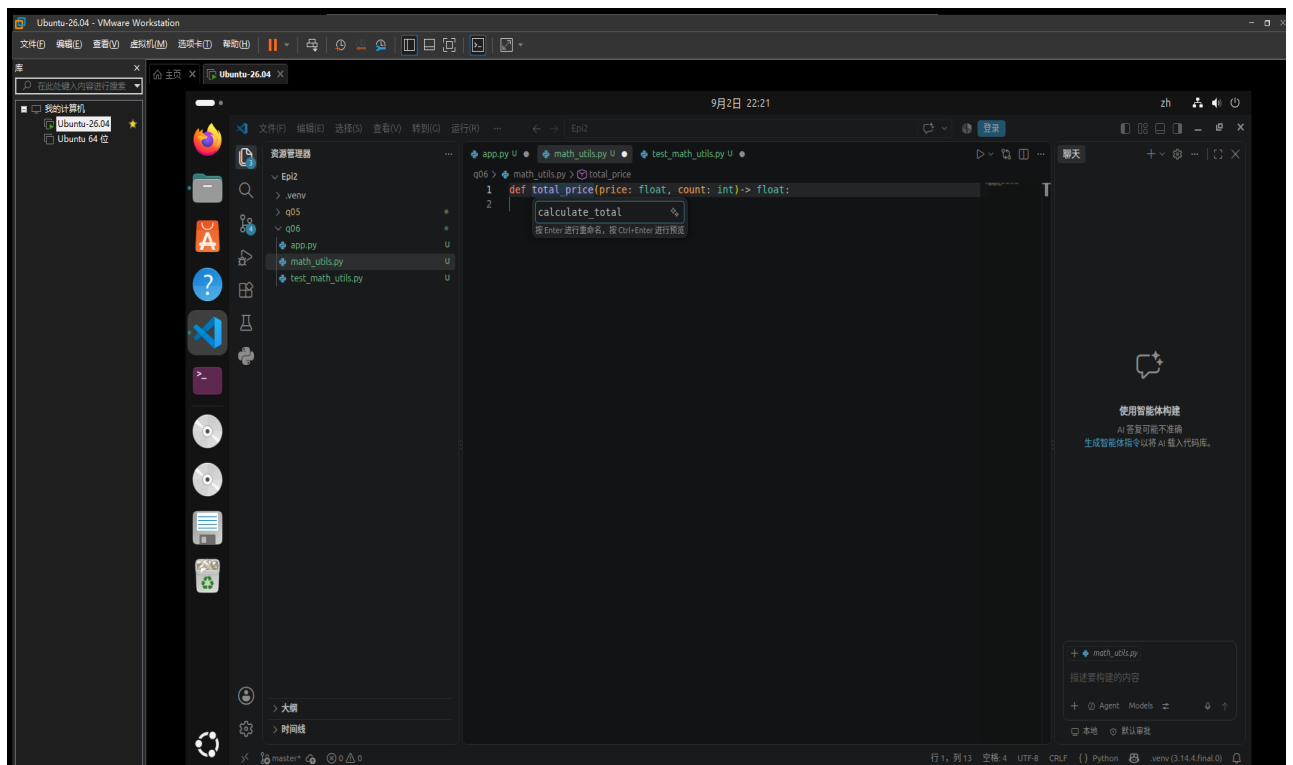
无

四、程序运行结果

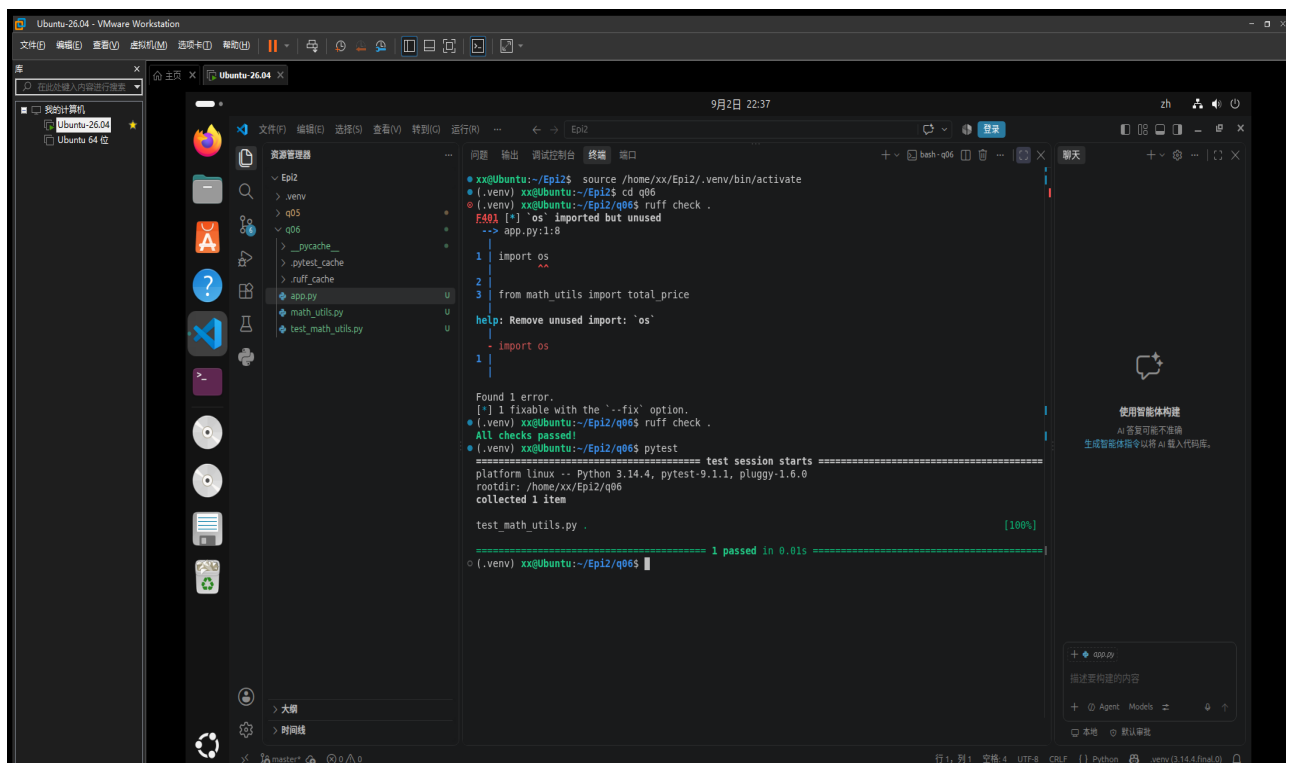
转到定义和查找引用：



重命名符号：



改错和 pytest:



五、解题感悟

1. 这次第一次比较完整地用到了语言服务器的功能，发现跳转定义和查找引用在代码多起来以后会方便很多。
2. 直接修改函数名容易漏掉其他地方的引用，使用重命名符号可以一起修改，感觉比手动搜索更可靠。

3. ruff 和 pytest 一个检查代码问题，一个检查程序功能，最后一起运行可以更早发现问题，平时写代码也可以养成这个习惯。

第 7 题：用调试器定位归并排序缺陷

一、主题

Debugging

二、题目要求

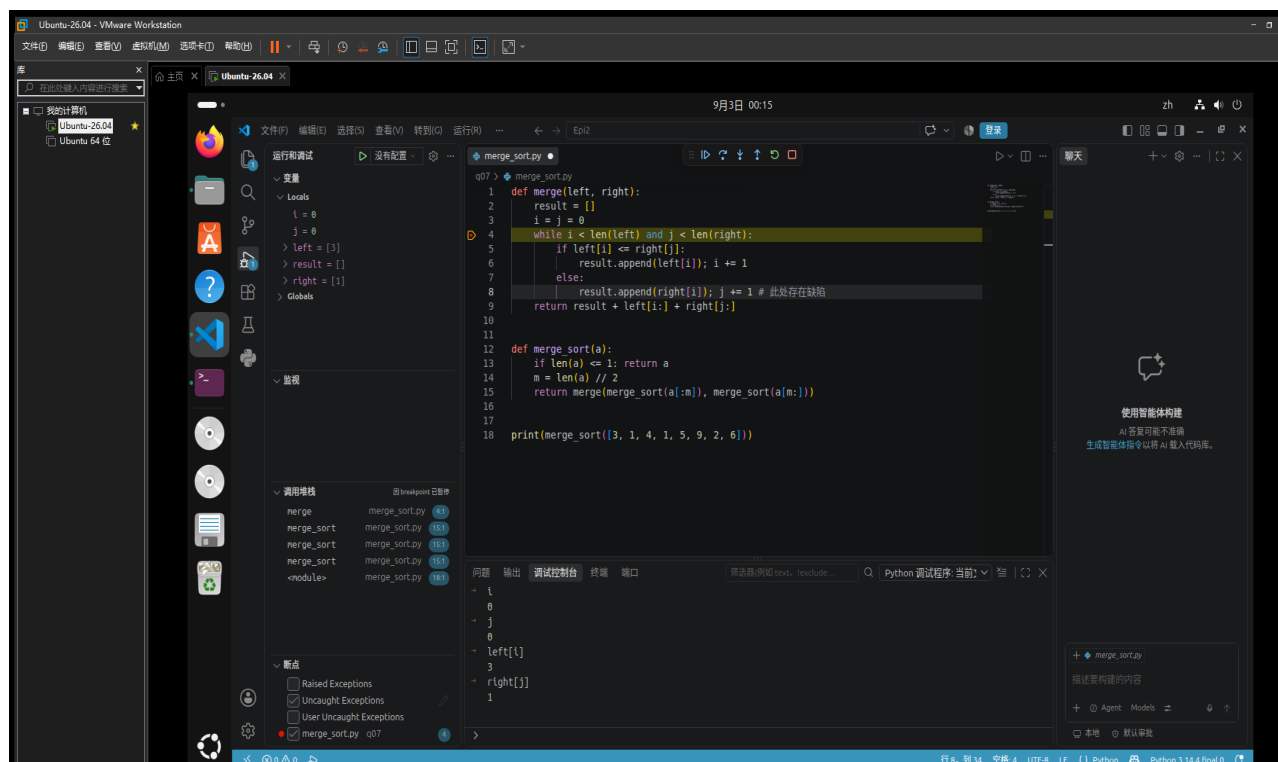
1. 将给定的归并排序代码保存到 q07/merge_sort.py，先运行给定输入，确认程序结果错误或出现异常。
2. 使用 pdb、IDE Debugger 或其他调试器，在 merge 函数中设置断点，并观察 i、j、left[i] 和 right[j] 的变化。
3. 找到代码中的错误并进行最小修改，不得直接使用 sorted 替换归并排序。
4. 使用 pytest 增加两个测试，分别测试题目给定的输入和含有重复元素的列表，并保证测试全部通过。

三、源程序代码

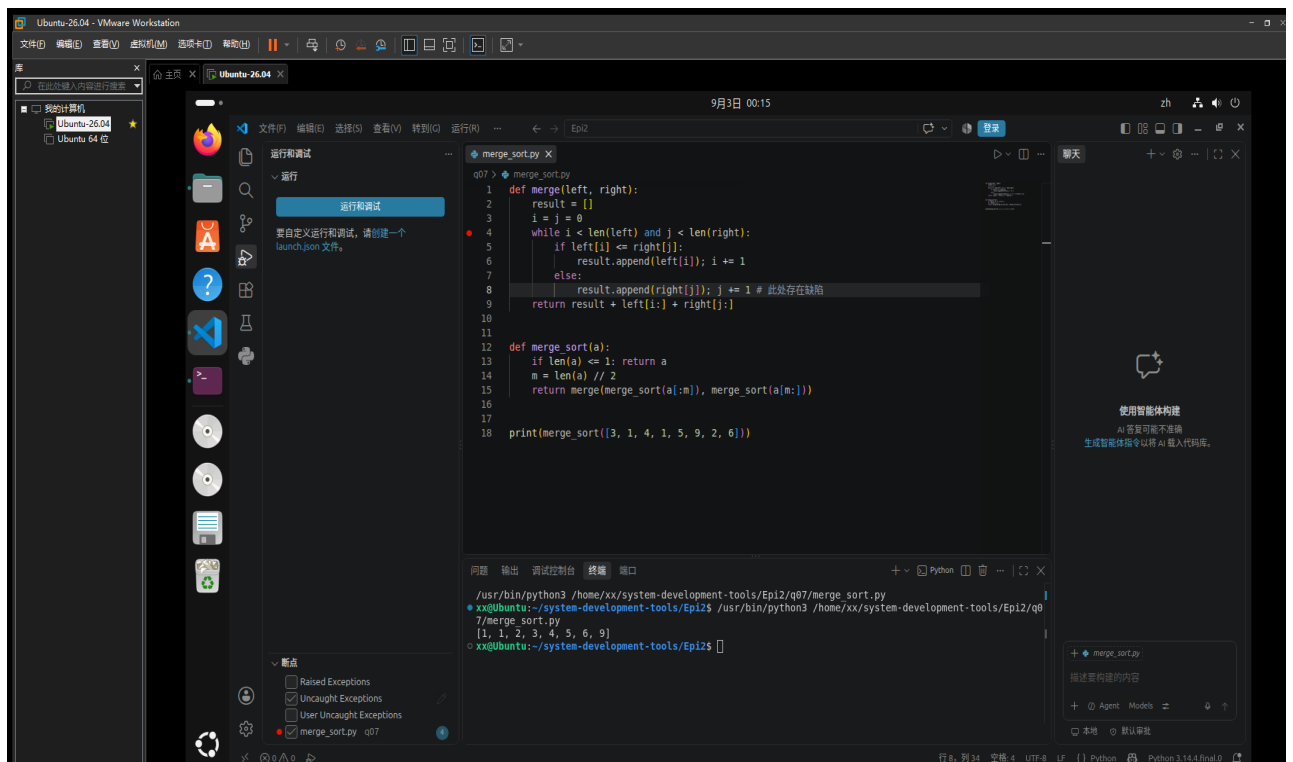
无

四、程序运行结果

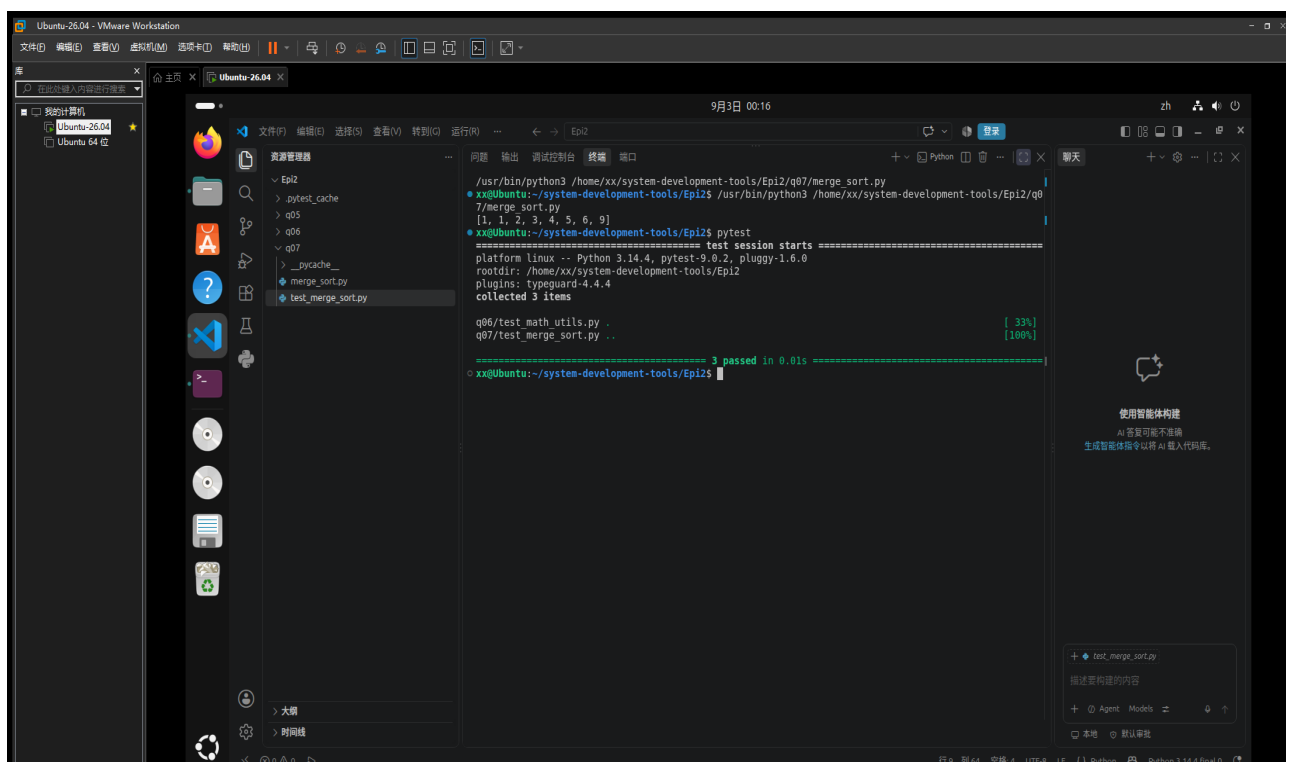
debug:



vscode 运行结果：



pytest 结果：



五、解题感悟

1. 直接看代码的时候不太容易发现下标的问题，运行调试器后观察 i 和 j 的变化就比较容易定位错误。
2. 这次体会到断点调试比单纯加很多 `print` 更方便，可以直接看到程序运行到某一行时变量的具体状态。

3. 修复后再用 pytest 测试，比只运行一次程序更放心，也能检查包含重复元素的情况有没有问题。

第 8 题：先测量，再优化慢速词频程序

一、主题

Profiling

二、题目要求

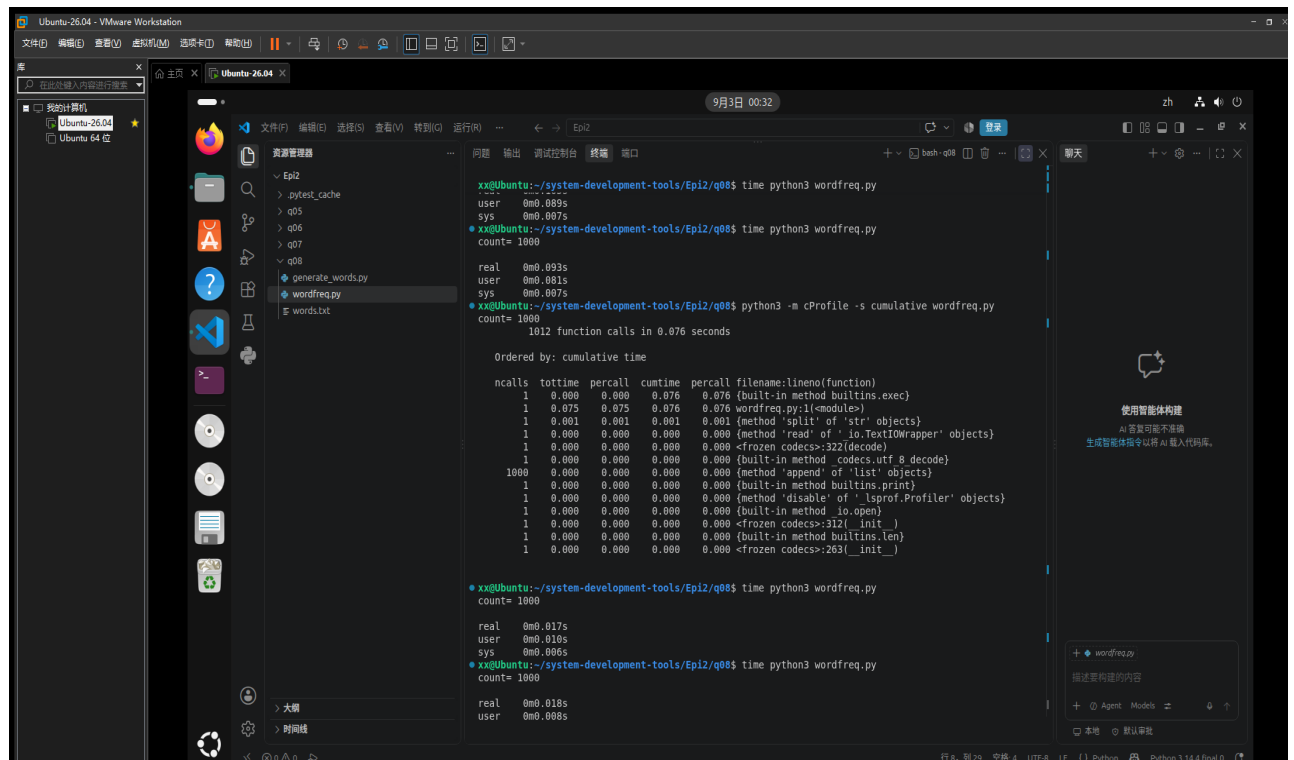
1. 在 q08 中创建 generate_words.py 和 wordfreq.py，运行生成程序得到固定的 words.txt。
2. 使用 time 运行原始词频程序两次，记录每次运行耗时。
3. 使用 cProfile -s cumulative 运行一次程序，根据分析结果找出主要耗时位置。
4. 在不改变最终 count 的情况下优化程序，不能直接把 1000 写死。
5. 使用相同的 words.txt 再运行优化后的程序两次，记录耗时，并计算优化前后的中位数和加速比。

三、源程序代码

无

四、程序运行结果

vscode 结果：



```
xx@Ubuntu:~/system-development-tools/Epi2/q08$ time python3 wordfreq.py
user    0m0.089s
sys      0m0.007s
xx@Ubuntu:~/system-development-tools/Epi2/q08$ time python3 wordfreq.py
count= 1000
real    0m0.093s
user    0m0.081s
sys      0m0.007s
xx@Ubuntu:~/system-development-tools/Epi2/q08$ python3 -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.076 seconds

Ordered by: cumulative time
ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1       0.000    0.000    0.076    0.076 {built-in method builtins.exec}
1       0.075    0.075    0.076    0.076 wordfreq.py:1(<module>)
1       0.001    0.001    0.001    0.001 {method 'split' of 'str' objects}
1       0.000    0.000    0.000    0.000 {method 'read' of 'io.TextIOWrapper' objects}
1       0.000    0.000    0.000    0.000 <frozen codecs>:322(decode)
1       0.000    0.000    0.000    0.000 {built-in method codecs.utf_8_decode}
1000     0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
1       0.000    0.000    0.000    0.000 {built-in method builtins.print}
1       0.000    0.000    0.000    0.000 {method 'disable' of 'lsprof.Profiler' objects}
1       0.000    0.000    0.000    0.000 {built-in method io.open}
1       0.000    0.000    0.000    0.000 <frozen codecs>:312(_init_)
1       0.000    0.000    0.000    0.000 {built-in method builtins.len}
1       0.000    0.000    0.000    0.000 <frozen codecs>:263(_init_)

xx@Ubuntu:~/system-development-tools/Epi2/q08$ time python3 wordfreq.py
count= 1000
real    0m0.017s
user    0m0.010s
sys      0m0.006s
xx@Ubuntu:~/system-development-tools/Epi2/q08$ time python3 wordfreq.py
count= 1000
real    0m0.018s
user    0m0.008s
```

项目优化前优化后

第一次 0.105 s 0.017 s

第二次 0.093 s 0.018 s

中位数 0.099 s 0.0175 s

count 1000 1000

加速比—约 5.66×

五、解题感悟

1. 这次发现程序能正常运行不代表效率就好，原程序虽然结果正确，但是使用列表查找重复单词会花费很多时间。
2. 通过 cProfile 可以先找到程序比较慢的地方，再决定怎么修改，而不是凭感觉乱优化。
3. 优化后 count 还是 1000，但运行时间明显降低，说明使用合适的数据结构对程序效率影响还是很大的。

课后练习

练习 1：处理以 - 开头的文件名

一、主题

命令行环境

二、题目要求

你可能见过像 `cmd --flag -- --notafalg` 这样的命令。这里的 `--` 是一个特殊参数，它告诉程序后面不要再继续解析选项（flag）了。也就是说，`--` 后面的所有内容都会被当作位置参数（positional argument）。这有什么用？试着运行 `touch -- -myfile`，然后在不使用 `--` 的情况下把它删掉。

三、源程序代码

```
touch -- -myfile
```

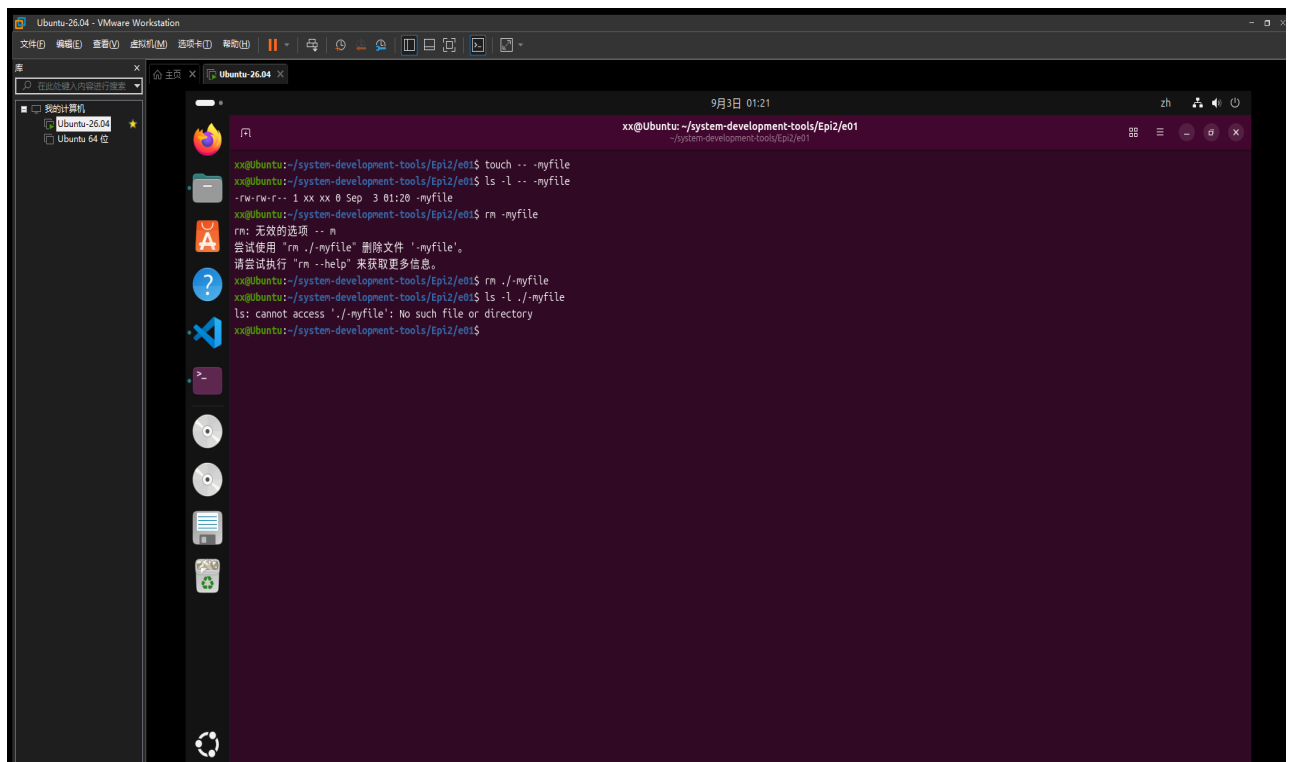
```
ls -l -- -myfile
```

```
rm -myfile
```

```
rm ./-myfile
```

```
ls -l ./-myfile
```

四、程序运行结果



五、解题感悟

1. 以前没注意到文件名以 - 开头时会被当成命令选项。
2. -- 可以让命令停止解析后面的选项。
3. 使用 ./-myfile 也可以正常处理这种文件名。

练习 2：组合 ls 参数查看文件信息

一、主题

命令行环境

二、题目要求

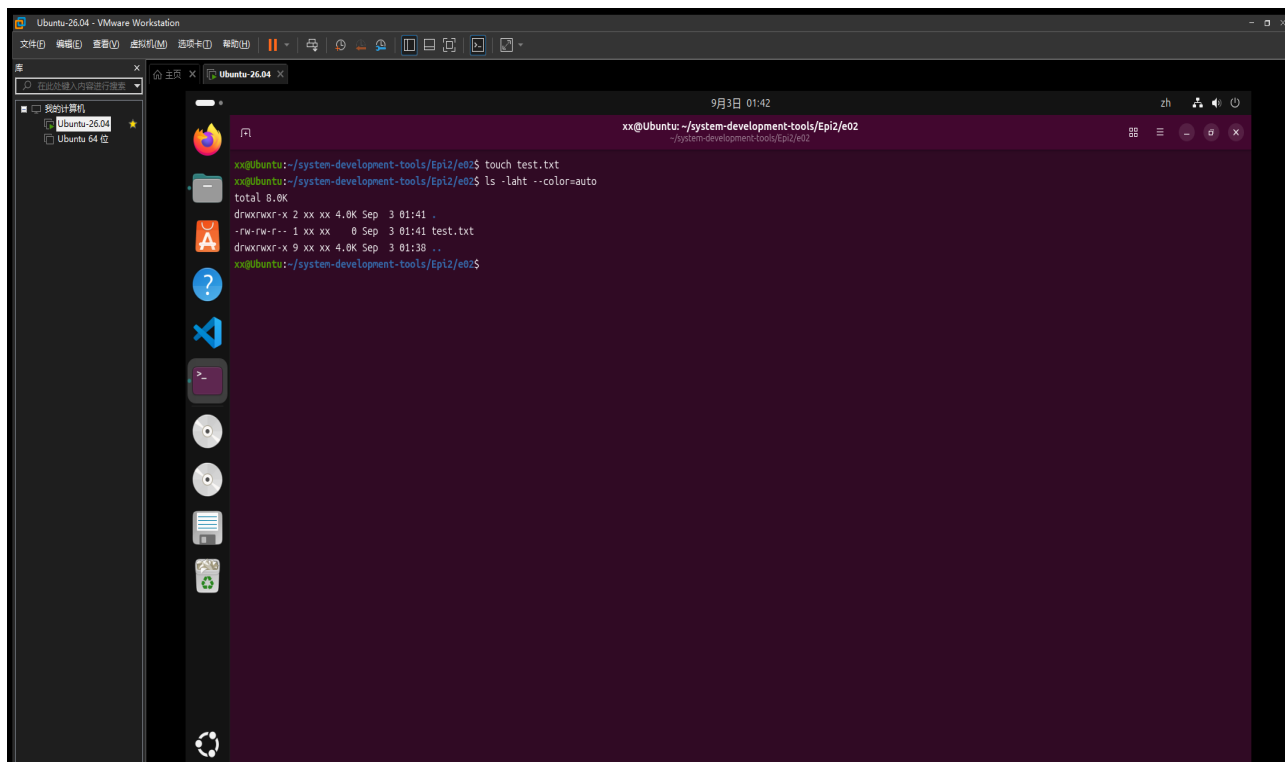
编写一条命令，列出当前目录中的所有文件，包括隐藏文件，以人类可读的格式显示，按照修改时间排序，并使用彩色输出。

三、源程序代码

```
touch test.txt
```

```
ls -laht --color=auto
```

四、程序运行结果



五、解题感悟

1. 这题主要是把几个 ls 参数组合起来使用。
2. -a 可以显示平时看不到的隐藏文件。
3. -t 可以按照修改时间排序，-h 让文件大小更容易看懂。

练习 3：为 cd 创建别名

一、主题

命令行环境

二、题目要求

创建一个别名，使运行 dc 时能够执行 cd，以防止不小心将 cd 输入成 dc。

三、源程序代码

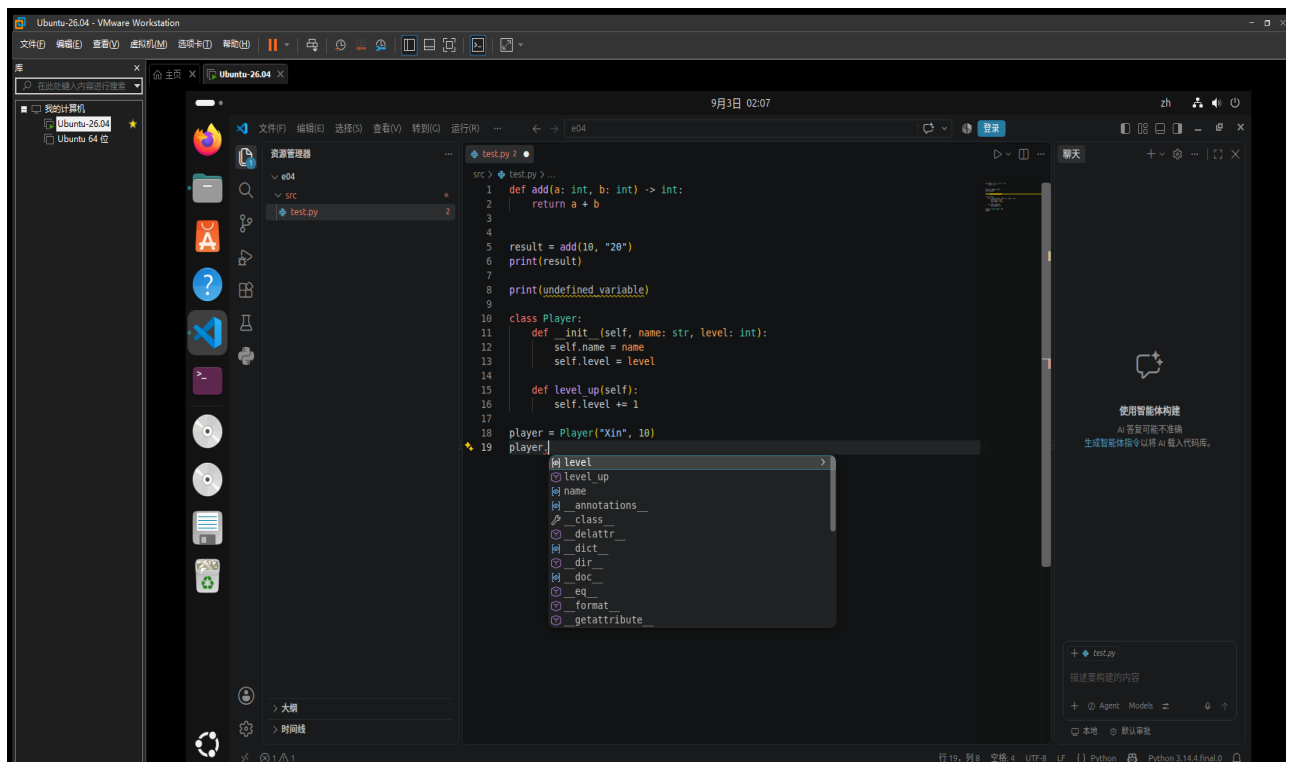
```
alias dc=cd
```

```
pwd
```

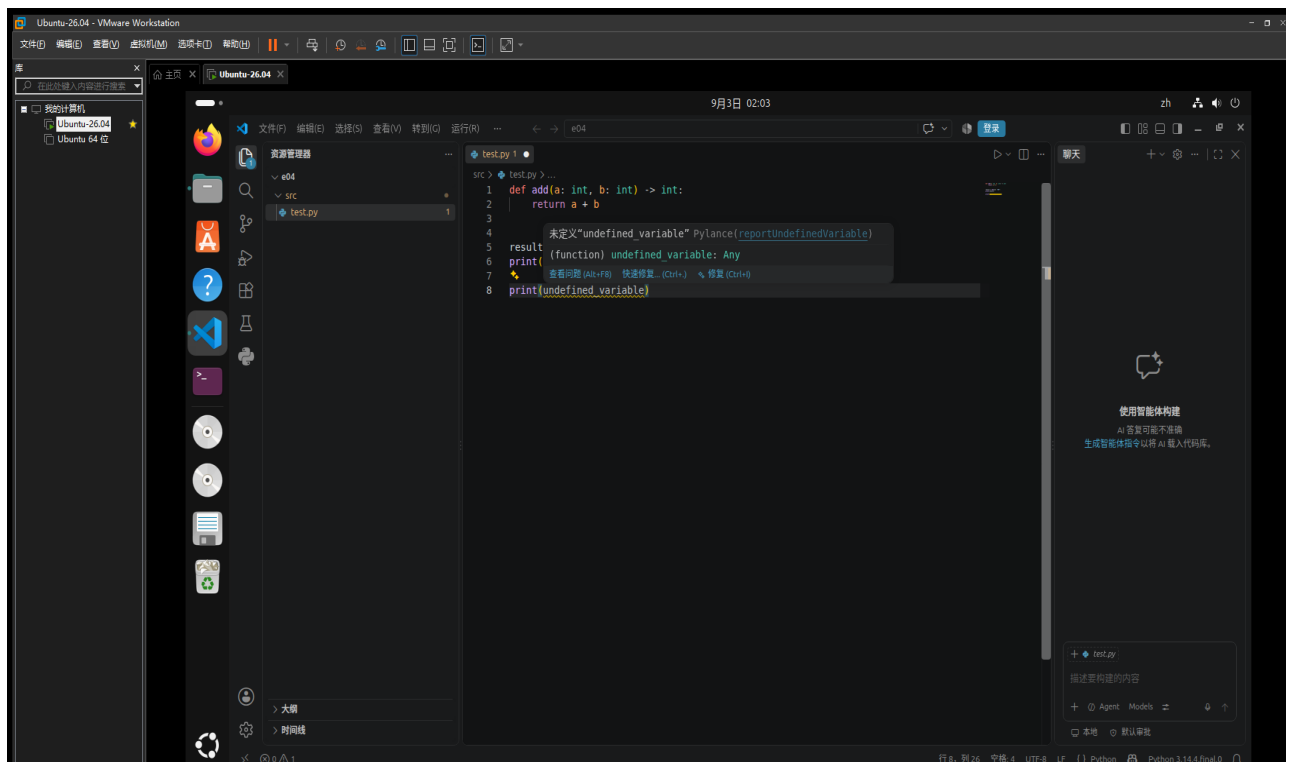
```
dc ..
```

```
pwd
```

四、程序运行结果



鼠标悬停问题检测：



五、解题感悟

1. 这次配置了 Python 的 IDE 扩展和 Pylance 语言服务器。
2. 体验了代码补全、类型检查和错误提示等功能。
3. 语言服务器可以帮助 IDE 更好地理解代码结构，提高编程效率。

练习 5：安装实用的 IDE 扩展

一、主题

开发环境与工具

二、题目要求

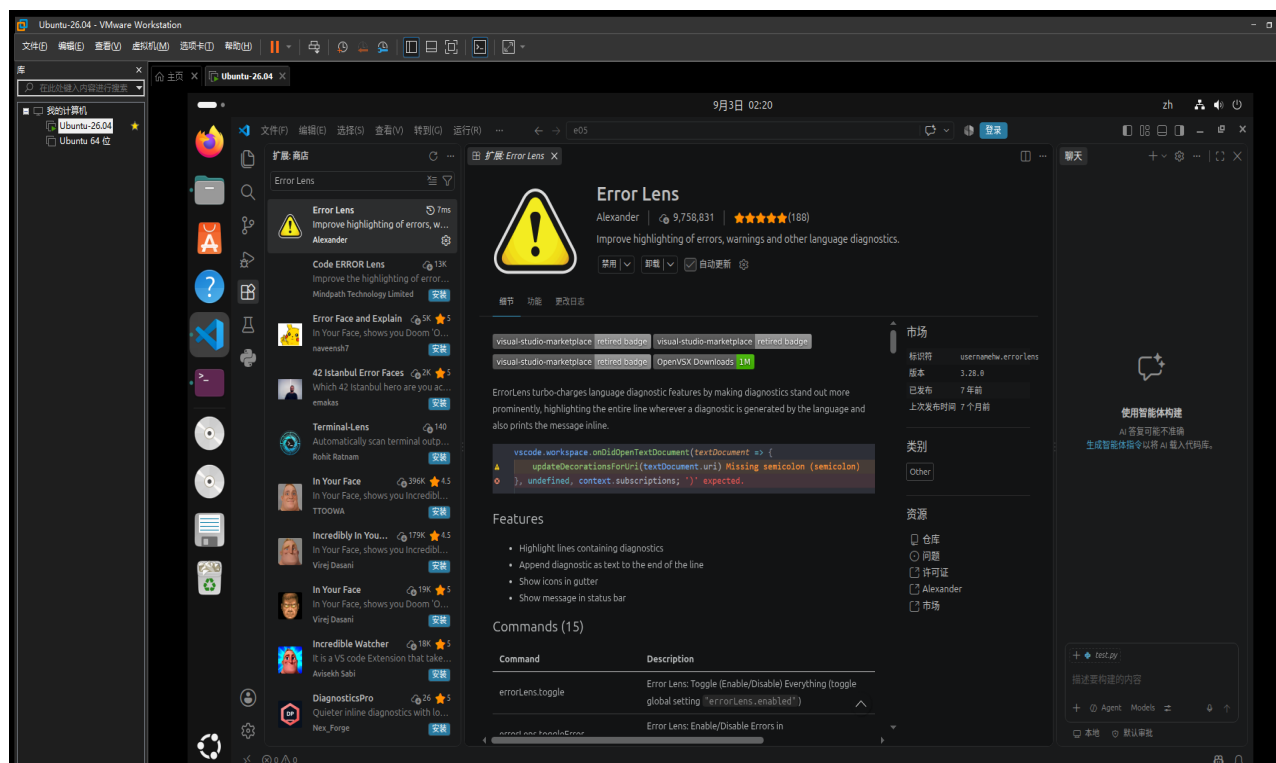
浏览一份 IDE 扩展列表，安装一个看起来对你有用的扩展。

三、源程序代码

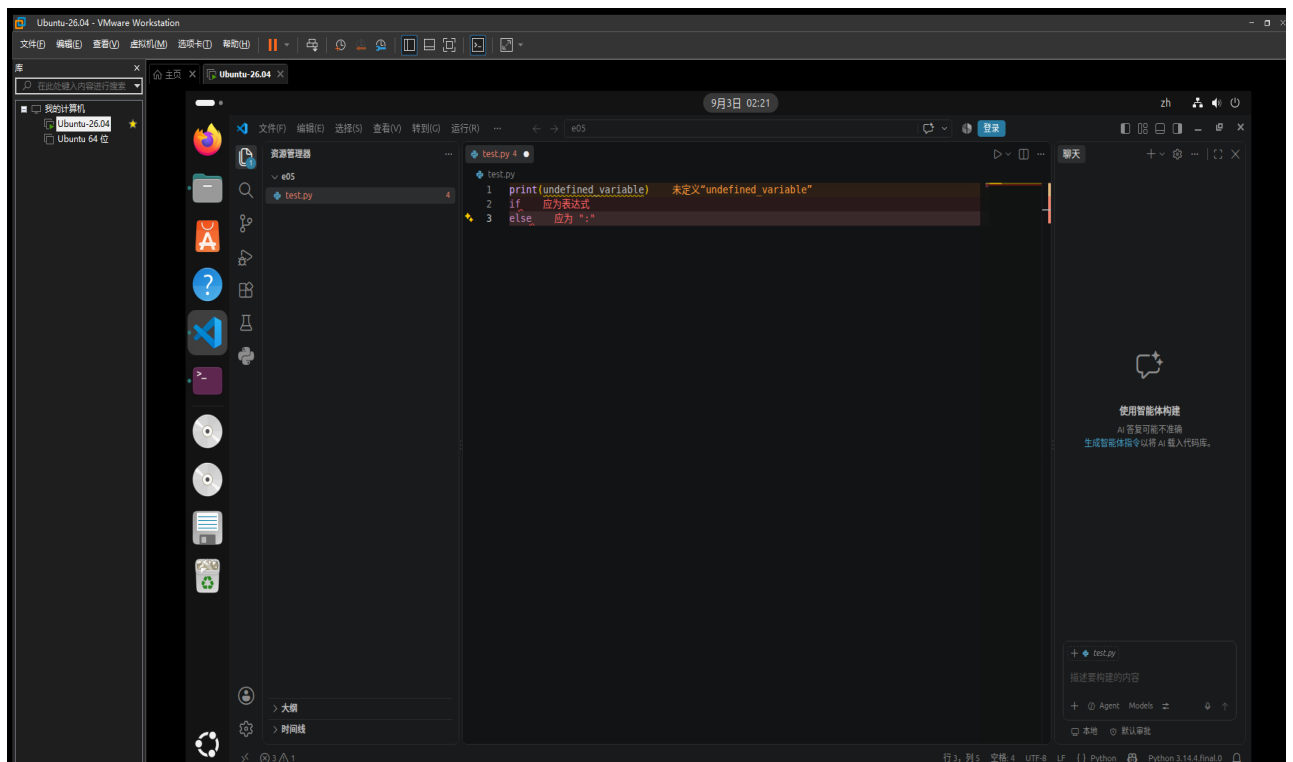
无

四、程序运行结果

安装拓展：



拓展效果：



五、解题感悟

1. 这次浏览了 VS Code 的扩展列表，并选择了 Error Lens。
2. Error Lens 可以直接在代码旁边显示错误和警告信息。
3. 安装扩展后，IDE 的使用体验更加方便。

练习 6：VimGolf——Even and Odd

一、主题

开发环境与工具

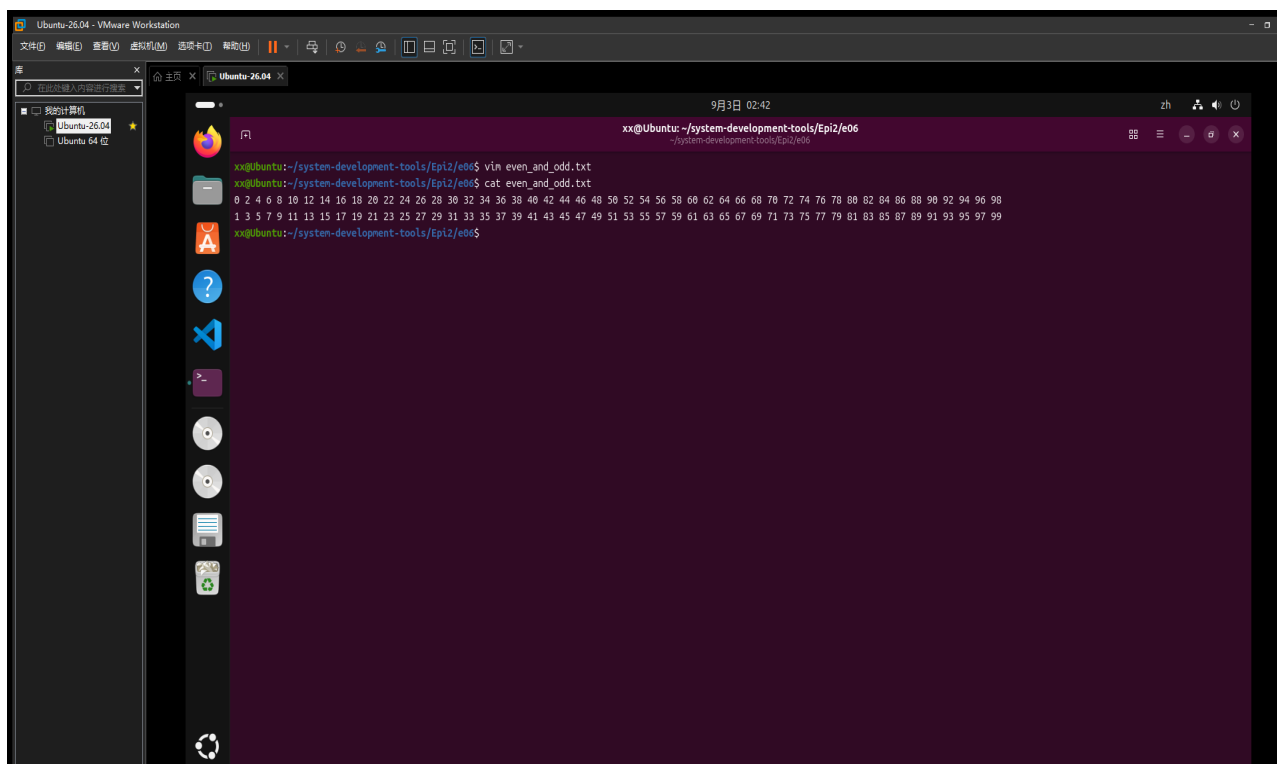
二、题目要求

将文件中的数字 0 到 99 重新排列为两行：第一行包含所有偶数，第二行包含所有奇数。每行中的数字按照从小到大的顺序排列，并以空格分隔。

三、源程序代码

```
:%d
:put =join(range(0,98,2))
:put =join(range(1,99,2))
:wq
```

四、程序运行结果



五、解题感悟

1. 这次练习了 Vim 的命令模式和表达式寄存器。
2. range() 可以快速生成指定范围和步长的数字。
3. 用 Vim 的命令处理大量文本比手动修改快很多。

练习 7：使用 AddressSanitizer 调试内存错误

一、主题

调试与性能分析

二、题目要求

使用 AddressSanitizer (ASan) 检测程序中的内存错误。编译并运行存在内存错误的程序，观察 AddressSanitizer 给出的错误信息，并根据错误提示定位和修复问题。

三、源程序代码

```
vim uaf.c  
gcc -g uaf.c -o uaf  
./uaf  
gcc -fsanitize=address -g uaf.c -o uaf_asan  
./uaf_asan  
vim uaf.c  
gcc -fsanitize=address -g uaf.c -o uaf_asan  
./uaf_asan
```

四、程序运行结果

```
xx@Ubuntu:~/system-development-tools/Epiz/e07$ vin uaf.c
xx@Ubuntu:~/system-development-tools/Epiz/e07$ gcc -g uaf.c -o uaf
./uaf
Hello
xx@Ubuntu:~/system-development-tools/Epiz/e07$ gcc -fsanitize=address -g uaf.c -o uaf_asan
xx@Ubuntu:~/system-development-tools/Epiz/e07$ ./uaf_asan
Hello
=====
==36555==ERROR: AddressSanitizer: heap-use-after-free on address 0x762ee77e0010 at pc 0x60d8c03301c3 bp 0x7ffffcfcba0 sp 0x7ffffcfcba0
WRITE of size 1 at 0x762ee77e0010 thread T0
#0 0x60d8c03301c4 in main /home/xx/system-development-tools/Epiz/e07/uaf.c:17
#1 0x7a0ee042a000 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:59
#2 0x7a0ee042a717 in __libc_start_main_impl ../csu/libc-start.c:360
#3 0x60d8c0330124 in _start (/home/xx/system-development-tools/Epiz/e07/uaf_asan+0x1124) (BuildId: 1a7c6c3e1bdc1c416f3d12d0be2dd7660780c6a0)

0x762ee77e0010 is located 0 bytes inside of 6-byte region [0x762ee77e0010,0x762ee77e0016)
freed by thread T0 here:
#0 0x7a0ee092a3ff in free ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:51
#1 0x60d8c0330390 in main /home/xx/system-development-tools/Epiz/e07/uaf.c:15
#2 0x7a0ee042a000 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:59
#3 0x7a0ee042a717 in __libc_start_main_impl ../csu/libc-start.c:360
#4 0x60d8c0330124 in _start (/home/xx/system-development-tools/Epiz/e07/uaf_asan+0x1124) (BuildId: 1a7c6c3e1bdc1c416f3d12d0be2dd7660780c6a0)

previously allocated by thread T0 here:
#0 0x7a0ee092b06f in malloc ../../../../src/libsanitizer/asan/asan_malloc_linux.cpp:67
#1 0x60d8c03301fe in main /home/xx/system-development-tools/Epiz/e07/uaf.c:5
#2 0x7a0ee042a000 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:59
#3 0x7a0ee042a717 in __libc_start_main_impl ../csu/libc-start.c:360
#4 0x60d8c0330124 in _start (/home/xx/system-development-tools/Epiz/e07/uaf_asan+0x1124) (BuildId: 1a7c6c3e1bdc1c416f3d12d0be2dd7660780c6a0)

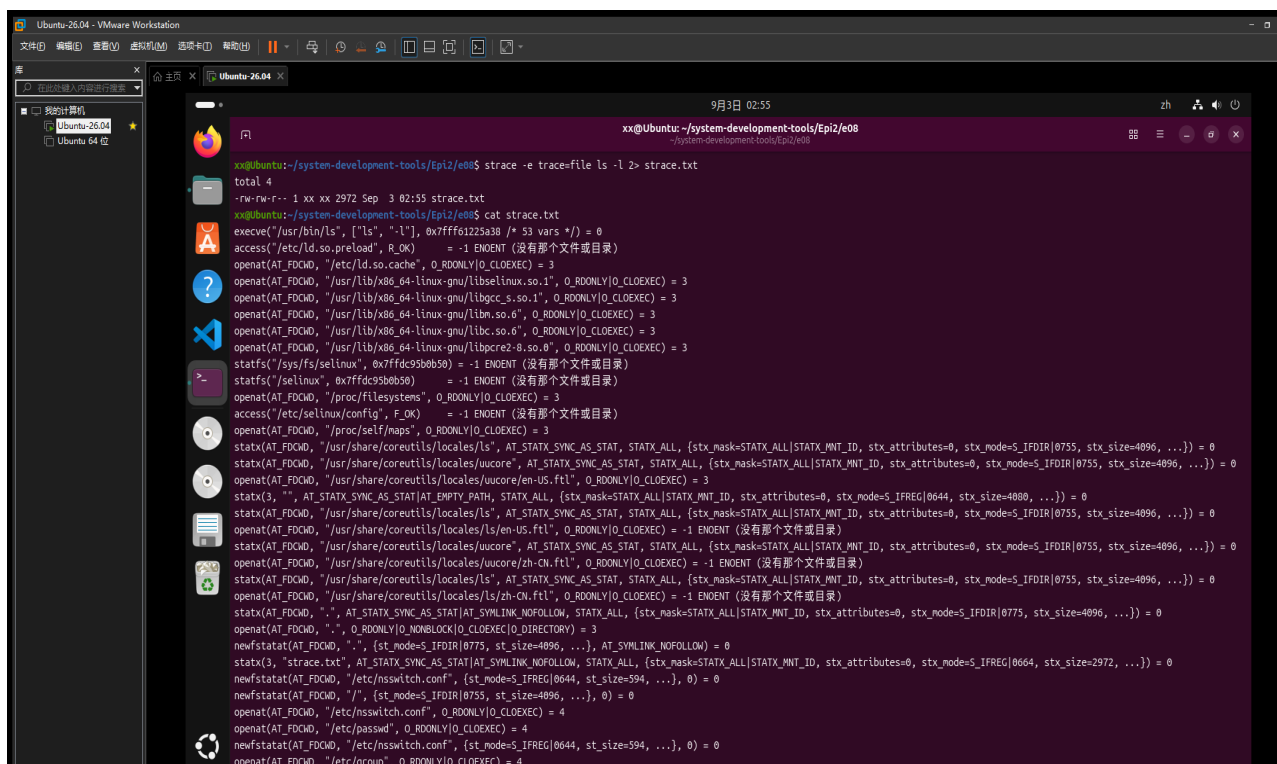
SUMMARY: AddressSanitizer: heap-use-after-free /home/xx/system-development-tools/Epiz/e07/uaf.c:17 in main
Shadow bytes around the buggy address:
 0x762ee77dfdb0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x762ee77dfdb8: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x762ee77dfdb0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x762ee77dfdb8: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 0x762ee77dfdb0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

```
0x762ee77dfdb0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x762ee77dfdb8: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x762ee77dfdb0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x762ee77dfdb8: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x762ee77dfdb0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x762ee77dfdb8: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x762ee77e0080: fa fa[fd] fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x762ee77e0088: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x762ee77e0090: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x762ee77e0098: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x762ee77e00a0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x762ee77e00b0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x762ee77e00c0: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone: fa
Freed heap region: fd
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: fe
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
Asan internal: fe
Left alloca redzone: ca
Right alloca redzone: cb
==36555==ABORTING
xx@Ubuntu:~/system-development-tools/Epiz/e07$ vin uaf.c
xx@Ubuntu:~/system-development-tools/Epiz/e07$ gcc -fsanitize=address -g uaf.c -o uaf_asan
xx@Ubuntu:~/system-development-tools/Epiz/e07$ ./uaf_asan
Hello
xx@Ubuntu:~/system-development-tools/Epiz/e07$
```

五、解题感悟

1. 这次学会了使用 -fsanitize=address 检查内存错误。
2. AddressSanitizer 可以直接指出发生错误的代码位置。
3. 通过这次练习发现，释放内存后继续使用指针会导致严重的问题。

练习 8：使用 strace 跟踪系统调用



五、解题感悟

1. 这次第一次使用 strace 查看程序的系统调用。
2. 发现一个简单的 ls 命令实际上会调用很多系统调用。
3. strace 可以帮助定位程序与操作系统之间的交互过程。

练习 9：使用大语言模型辅助调试

一、主题

调试与性能分析

二、题目要求

使用一个大语言模型（LLM）帮助调试一个难以理解的错误信息。可以尝试复制一条编译器错误信息（尤其是来自 C++ 模板或 Rust 的错误），并询问大语言模型如何解释和修复该错误。也可以尝试将 strace 或 AddressSanitizer 输出的一部分提供给大语言模型进行分析。

三、源程序代码

无

四、程序运行结果

llm 对话：



五、解题感悟

1. 这次尝试使用大语言模型分析程序中的错误信息。
2. LLM 可以帮助快速理解复杂的报错信息并定位问题。
3. 但还是需要自己理解代码并验证修改是否正确。

练习 10：使用 perf stat 获取性能统计信息

一、主题

调试与性能分析

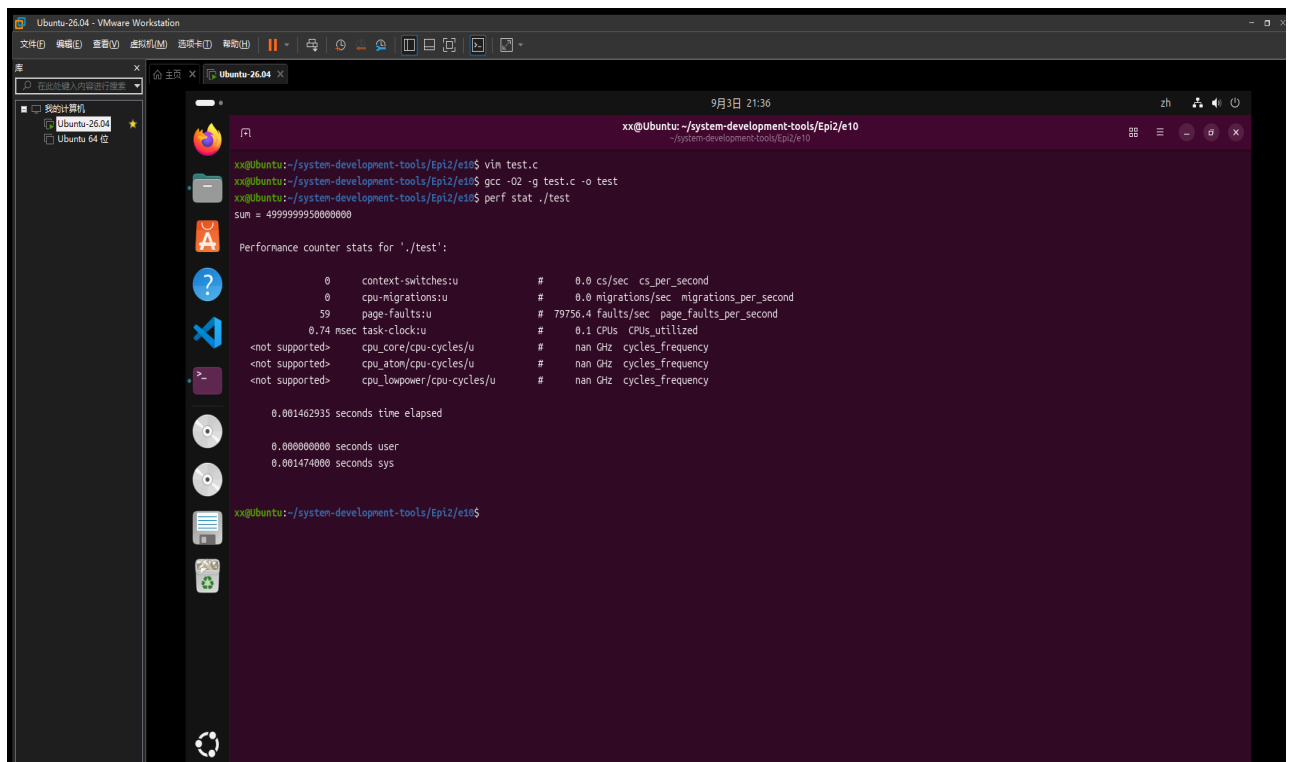
二、题目要求

使用 perf stat 获取一个程序的基本性能统计信息，并说明不同计数器的含义。

三、源程序代码

```
vim test.c
gcc -O2 -g test.c -o test
perf stat ./test
```

四、程序运行结果



五、解题感悟

1. 这次学习了使用 perf stat 查看程序的性能数据。
2. 了解了上下文切换、CPU 迁移和缺页异常等计数器的含义。
3. 通过这次实验发现，虚拟机环境可能不支持部分 CPU 硬件性能计数器。

练习 11：使用 hyperfine 进行性能测试

一、主题

调试与性能分析

二、题目要求

使用 hyperfine 对一个程序进行性能测试，观察程序的运行时间，并根据测试结果分析程序的性能。

三、源程序代码

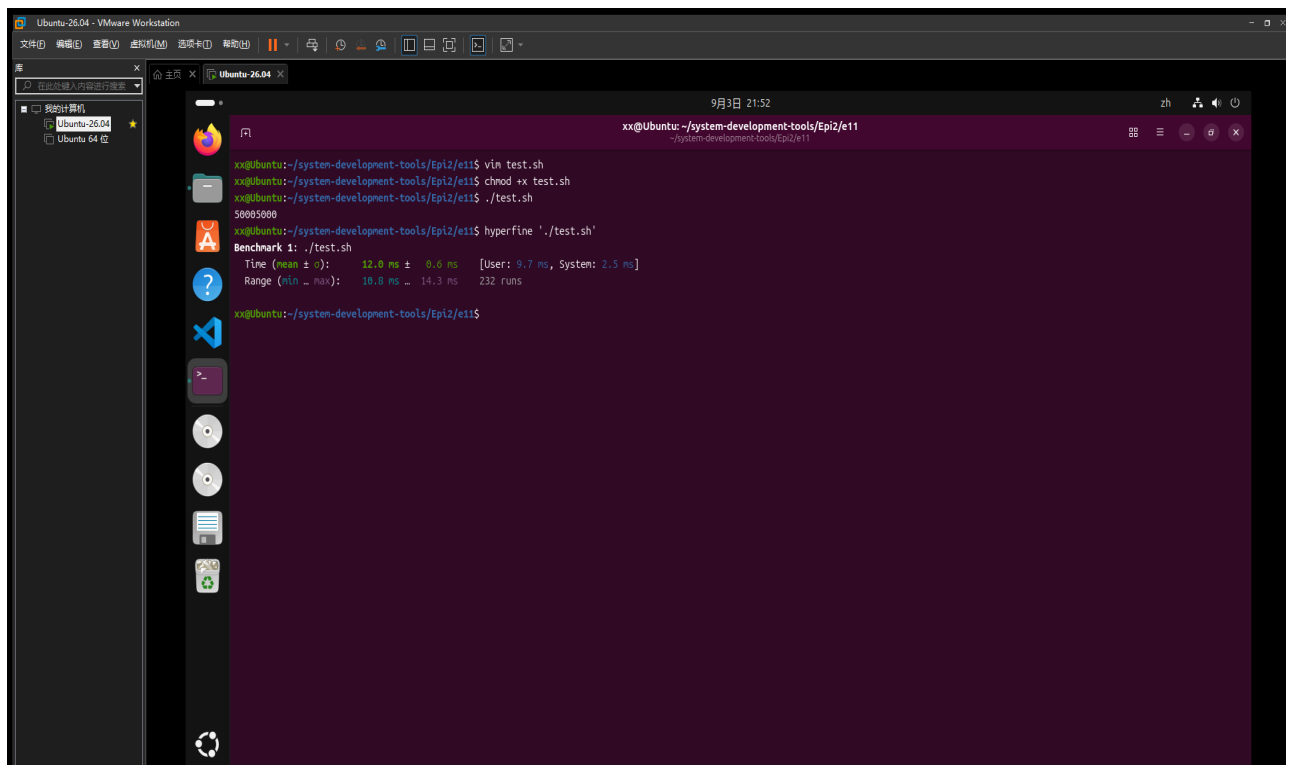
```
vim test.sh
```

```
chmod +x test.sh
```

```
./test.sh
```

```
hyperfine './test.sh'
```

四、程序运行结果



五、解题感悟

1. 这次学习了使用 hyperfine 自动进行多次性能测试。
2. 通过平均时间和标准差可以了解程序运行速度和稳定性。
3. 多次运行可以减少单次运行时间波动对测试结果的影响。

练习 12：查找占用 4444 端口的进程

一、主题

调试与性能分析

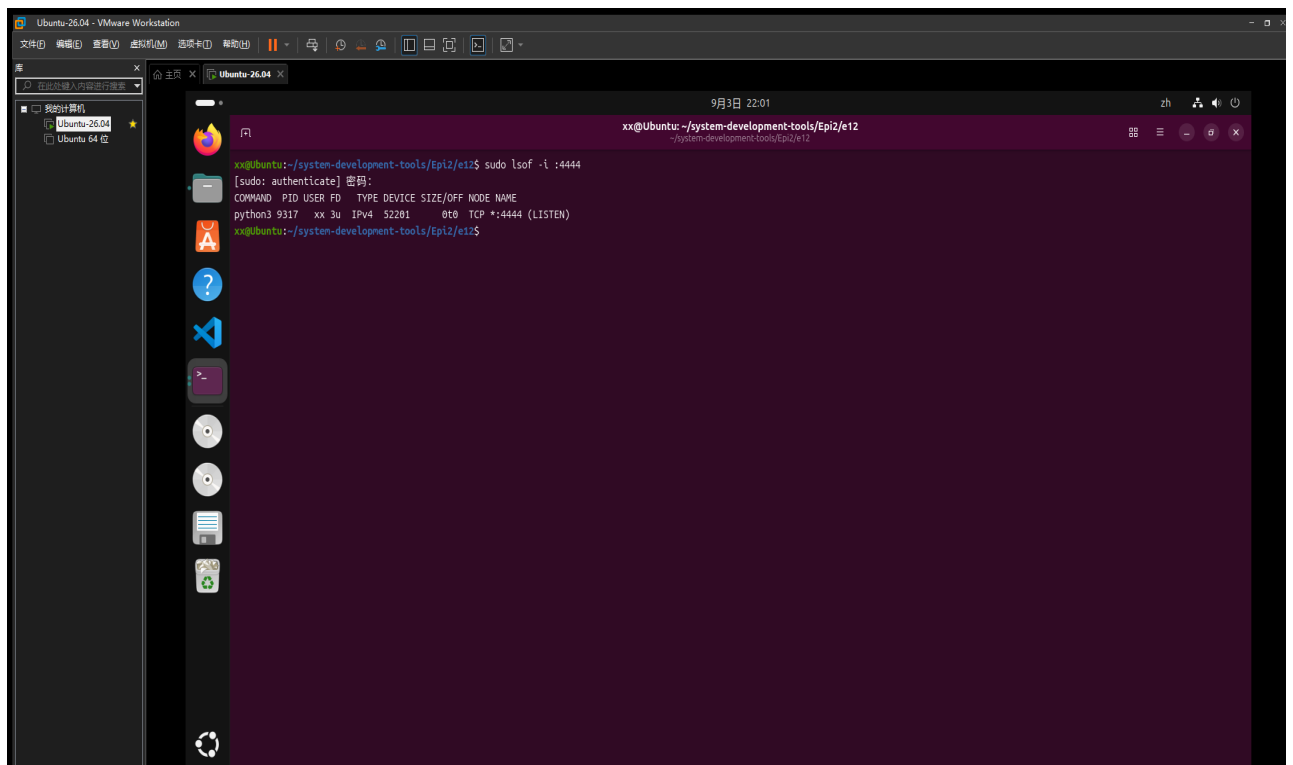
二、题目要求

查找当前正在占用 4444 端口的进程，并确定是哪个程序正在监听该端口。

三、源程序代码

```
sudo lsof -i :4444
```

四、程序运行结果



五、解题感悟

1. 这次学习了使用 lsof 查看端口被哪个进程占用。
2. PID 可以用来确定具体的进程。
3. 通过 `python3 -m http.server 4444` 创建测试服务后，可以验证端口查询命令是否正常。